

From: <https://chatgpt.com/c/6a8a2959-22ac-83ee-b837-2220612a953b>

you asked: go throgh

message time: 2026-08-23 04:25:22

go throgh the previous chats from start to end completly which are attached files and responce how detailed- <https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93> , <https://chatgpt.com/c/6a8a1a24-1218-83e9-9f55-db7ec639b9a5> , <https://chatgpt.com/c/6a8a20f0-f3fc-83ee-891b-ba811975393f> , <https://chatgpt.com/c/6a8a2356-f96c-83ee-950e-82f71a5e00eb> , <https://chatgpt.com/c/6a8a251a-31f8-83e8-9107-c2dfbcaab421> , <https://chatgpt.com/c/6a8a279f-3c40-83ee-8aa1-24574734f5ea>

And continue further according to previous - Module 1,2,3,4,5,6 completed as aboved thier links given and also attached files . Now continue with **Module 7**: 50 SQL questions + queries. For your goal, I recommend turning this into a structured “2-Year Java + Spring Boot Product Company Interview Bible” with: Module 1: 150 Core Java questions + answers Module 2: 120 Spring/Spring Boot questions + internal working Module 3: 80 JPA/Hibernate questions Module 4: 75 REST/API questions Module 5: 75 Microservices questions Module 6: 75 Spring Security/JWT questions Module 7: 50 SQL questions + queries Module 8: 50 Multithreading questions Module 9: 50 Kafka + Redis questions Module 10: 50 Docker/AWS questions Module 11: 100 DSA problems specifically for 2-year Java SDE interviews Module 12: 50 LLD/system-design scenarios Module 13: 100 project/resume follow-up questions Module 14: Company-specific patterns for Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc. Most importantly, for every question I'd structure it as: Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant That format will train you to answer follow-up questions instead of just memorizing definitions.

chatgpt response: I'll use t

I'll use the uploaded Module 5/6 exports as the continuity source, preserve the exact interview format and depth, and start Module 7 at Q1 without restarting earlier modules.

Absolutely. I've preserved the continuity from the uploaded Module 5 and Module 6 exports rather than restarting the curriculum.

The previous material explicitly established the master-bible sequence and the exact format:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up → Tricky interviewer counter-question → Best answer → Code/diagram where relevant.

It also establishes that Module 5 ended at the production-grade microservices system-design level and Module 6 continued into Spring Security/JWT internals.

□ Module 7 — SQL

50 Interview Questions + Queries

2-Year Java + Spring Boot Product-Company Interview Bible

For this module, I'll follow the same philosophy as the previous modules: not just definitions, but **query writing + reasoning + performance + real backend scenarios + interviewer traps**.

PART A — SQL FUNDAMENTALS

Q1–Q10

Q1. What is SQL? ☐

Short interview answer

SQL stands for **Structured Query Language** and is used to interact with relational databases for storing, retrieving, modifying, and controlling data.

Detailed explanation

SQL is commonly used for:

text

```
DDL → CREATE, ALTER, DROP
DML → INSERT, UPDATE, DELETE
DQL → SELECT
DCL → GRANT, REVOKE
TCL → COMMIT, ROLLBACK
```

Example:

sql

```
SELECT id, name, salary
FROM employee
WHERE salary > 50000;
```

Real project example

In a Spring Boot application:

text

```
Controller
    ↓
Service
    ↓
```

```
Repository
  ↓
Hibernate/JPA
  ↓
SQL
  ↓
Database
```

JPA may generate SQL internally, but understanding SQL remains critical for debugging performance.

Follow-up

Does JPA eliminate the need to know SQL?

No.

A developer still needs SQL for:

- performance analysis
- complex queries
- indexes
- joins
- native queries
- debugging
- database optimization

Tricky counter-question

Can SQL itself guarantee business correctness?

Not completely.

Business correctness is usually enforced through a combination of:

```
text
Application logic
+
Database constraints
+
Transactions
+
Concurrency control
```

Best answer

SQL is the language used to interact with relational databases. In backend development I use it not only for CRUD operations but also for joins, aggregation, transactions, indexing and performance optimization.

Q2. What is the difference between `WHERE` and `HAVING`? ☐☐☐

Short answer

`WHERE` filters **rows before grouping**, while `HAVING` filters **groups after aggregation**.

Example

Suppose:

text

employee

id	department	salary
----	------------	--------

1	IT	50000
---	----	-------

2	IT	70000
---	----	-------

3	HR	40000
---	----	-------

4	HR	45000
---	----	-------

5	SALES	80000
---	-------	-------

Find departments whose average salary is greater than 50,000.

sql

```
SELECT department, AVG(salary)
FROM employee
GROUP BY department
HAVING AVG(salary) > 50000;
```

We cannot normally write:

sql

```
WHERE AVG(salary) > 50000
```

because the aggregate is calculated after grouping.

Query execution mental model

Think:

text

FROM

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

SELECT

↓

ORDER BY

Tricky question

Can `WHERE` and `HAVING` be used together?

Yes.

sql

```
SELECT department, AVG(salary)
FROM employee
WHERE status = 'ACTIVE'
GROUP BY department
HAVING AVG(salary) > 50000;
```

Here:

text

WHERE

→ removes inactive employees

GROUP BY

→ creates department groups

HAVING

→ removes departments whose average salary is <= 50000

Best answer

WHERE filters individual rows before grouping, while HAVING filters grouped results after aggregation.

Q3. What is a Primary Key? □ □

Short answer

A primary key uniquely identifies each row in a table.

Example:

```
sql
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    salary DECIMAL(10,2)
);
```

Properties

A primary key is generally:

```
text
UNIQUE
NOT NULL
```

A table can have only one primary-key constraint, although it can contain multiple columns.

Composite primary key

```
sql
PRIMARY KEY (employee_id, project_id)
```

This means uniqueness is based on the combination.

Real project example

```
text
orders
-----
order_id  PK
customer_id
amount
status
```

`order_id` uniquely identifies an order.

Tricky question

Can a table have two primary keys?

No.

But one primary key may contain multiple columns.

Q4. Primary Key vs Unique Key? ☐ ☐

Primary Key	UNIQUE
Identifies row	Enforces uniqueness
One primary-key constraint	Multiple UNIQUE constraints possible
Cannot be NULL	NULL handling depends on DB
Main row identity	Candidate alternate key

Example:

```
sql
CREATE TABLE users (
    id BIGINT PRIMARY KEY,
    username VARCHAR(100) UNIQUE,
    email VARCHAR(200) UNIQUE
);
```

Here:

```
text
id → primary key
username → unique
email → unique
```

Interview trap

A unique constraint is not automatically equivalent to the primary key.

The primary key represents the table's primary identity.

Q5. What is a Foreign Key? ☐

Short answer

A foreign key references a key in another table and helps enforce referential integrity.

Example:

sql

```
CREATE TABLE customer (  
    id BIGINT PRIMARY KEY,  
    name VARCHAR(100)  
);  
  
CREATE TABLE orders (  
    id BIGINT PRIMARY KEY,  
    customer_id BIGINT,  
    amount DECIMAL(10,2),  
  
    FOREIGN KEY (customer_id)  
        REFERENCES customer(id)  
);
```

Meaning

text

```
customer  
  1  
  |  
  | customer_id  
  |  
  N  
orders
```

An order refers to an existing customer.

Why important?

It prevents invalid relationships.

For example, depending on constraint configuration, you cannot insert:

text

```
customer_id = 999999
```

when customer 999999 does not exist.

Q6. What is a JOIN? ☐ ☐ ☐

Short answer

A JOIN combines rows from multiple tables based on a related condition.

Example:

```
sql
SELECT
    o.id,
    c.name,
    o.amount
FROM orders o
JOIN customer c
    ON o.customer_id = c.id;
```

Mental model

```
text
orders
  +
customer
  ↓
combined result
```

Common joins

```
text
INNER JOIN
LEFT JOIN
RIGHT JOIN
FULL OUTER JOIN
CROSS JOIN
SELF JOIN
```

Q7. INNER JOIN vs LEFT JOIN? ☐☐☐

INNER JOIN

Returns only matching rows.

```
sql
SELECT c.id, c.name, o.id
FROM customer c
```

```
INNER JOIN orders o
  ON c.id = o.customer_id;
```

A customer with no order is excluded.

LEFT JOIN

Returns all rows from the left table, even if no match exists.

```
sql
SELECT c.id, c.name, o.id
FROM customer c
LEFT JOIN orders o
  ON c.id = o.customer_id;
```

Customer without orders:

```
text
customer_id | name | order_id
10          | John | NULL
```

Interview example

Find customers who have never placed an order.

```
sql
SELECT c.id, c.name
FROM customer c
LEFT JOIN orders o
  ON c.id = o.customer_id
WHERE o.id IS NULL;
```

☐ This is a very important interview pattern.

Q8. What is a self join?

Short answer

A self join joins a table with itself.

Example:

```
text
employee
-----
```

```
id
name
manager_id
```

Find employee-manager relationships:

```
sql
SELECT
    e.name AS employee,
    m.name AS manager
FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.id;
```

Diagram

```
text
employee e
    |
    | manager_id
    ↓
employee m
```

Real project

Hierarchical data such as:

```
text
CEO
├─ Engineering Manager
│   ├─ Developer
│   └─ Developer
└─ Sales Manager
```

Q9. What is a CROSS JOIN?

Short answer

A CROSS JOIN creates the Cartesian product of two tables.

Suppose:

text

A = 3 rows

B = 4 rows

Result:

text

3 × 4 = 12 rows

Example:

sql

SELECT *

FROM colors

CROSS JOIN sizes;

Real use case

Generating combinations:

text

Products

x

Sizes

or:

text

Dates

x

Stores

Tricky question

Why can CROSS JOIN become dangerous?

Because the number of rows can grow rapidly.

text

1000 × 1000

=

1,000,000

Q10. What is `GROUP BY`? □ □

Short answer

GROUP BY groups rows having the same values so aggregate functions can be applied.

Example:

```
sql
SELECT department,
       COUNT(*) AS employee_count
FROM employee
GROUP BY department;
```

Output:

```
text
IT      20
HR      10
SALES   15
```

Common aggregate functions

```
text
COUNT()
SUM()
AVG()
MIN()
MAX()
```

Follow-up

Find total salary per department:

```
sql
SELECT department,
       SUM(salary) AS total_salary
FROM employee
GROUP BY department;
```

PART B — IMPORTANT QUERY PATTERNS

Q11–Q20

Q11. How do you find the second-highest salary? ☐☐☐☐☐

This is one of the classic SQL interview questions.

Method 1 — `DISTINCT + ORDER BY + OFFSET`

For databases supporting the syntax:

```
sql
SELECT DISTINCT salary
FROM employee
ORDER BY salary DESC
OFFSET 1 ROW
FETCH NEXT 1 ROW ONLY;
```

Method 2 — Subquery

```
sql
SELECT MAX(salary)
FROM employee
WHERE salary < (
    SELECT MAX(salary)
    FROM employee
);
```

Why this is useful?

Suppose:

```
text
100000
90000
90000
80000
```

Second-highest **distinct** salary is:

```
text
90000
```

Tricky interviewer question

What if interviewer asks for second row rather than second distinct salary?

That's different.

You need to clarify whether duplicates count.

Q12. Find the Nth highest salary. ☐☐☐

A common approach:

```
sql
SELECT DISTINCT salary
FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

For third-highest:

```
text
OFFSET 2
```

For Nth:

```
text
OFFSET N - 1
```

In production SQL, syntax depends on database vendor.

Alternative with `DENSE\_RANK`

```
sql
SELECT salary
FROM (
    SELECT salary,
           DENSE_RANK() OVER (
               ORDER BY salary DESC
           ) AS rnk
    FROM employee
) t
WHERE rnk = 3;
```

This is more powerful because it handles duplicate salaries explicitly.

Q13. What is the difference between `RANK()`, `DENSE\_RANK()` and `ROW\_NUMBER()`? ☐☐☐☐

This is extremely important.

Suppose salaries:

text

100
100
90
80

ROW_NUMBER

text

100 → 1
100 → 2
90 → 3
80 → 4

Every row gets a unique number.

RANK

text

100 → 1
100 → 1
90 → 3
80 → 4

It leaves a gap after ties.

DENSE_RANK

text

100 → 1
100 → 1
90 → 2
80 → 3

No gap.

Query

sql

```
SELECT name,  
       salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,  
       RANK() OVER (ORDER BY salary DESC) AS rnk,
```



```
DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk  
FROM employee;
```

Interview memory

text

ROW_NUMBER → unique sequence
RANK → gaps after ties
DENSE_RANK → no gaps after ties

Q14. Find duplicate records. ☐☐☐

Suppose duplicate emails exist.

sql

```
SELECT email,  
       COUNT(*) AS count  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Why?

text

GROUP BY email
→ creates groups

HAVING COUNT(*) > 1
→ keeps only duplicates

Find duplicate usernames

sql

```
SELECT username  
FROM users  
GROUP BY username  
HAVING COUNT(*) > 1;
```

Tricky question

How do you delete duplicates while keeping one row?

A common approach uses `ROW_NUMBER()`.

```

sql
WITH duplicates AS (
    SELECT id,
           ROW_NUMBER() OVER (
               PARTITION BY email
               ORDER BY id
           ) AS rn
    FROM users
)
DELETE FROM users
WHERE id IN (
    SELECT id
    FROM duplicates
    WHERE rn > 1
);

```

Syntax can vary by database.

Q15. Find employees earning more than the average salary.

```

sql
SELECT *
FROM employee
WHERE salary > (
    SELECT AVG(salary)
    FROM employee
);

```

Explanation

Inner query:

```

sql
SELECT AVG(salary)
FROM employee

```

calculates average.

Outer query:

```

sql
WHERE salary > average

```

returns employees above it.

Follow-up

Can this be solved with a window function?

Yes:

```
sql
SELECT *
FROM (
    SELECT e.*,
           AVG(salary) OVER () AS avg_salary
    FROM employee e
) t
WHERE salary > avg_salary;
```

Q16. Find employees earning more than their department average. ☐☐☐☐

This is much more interview-relevant.

Window-function approach

```
sql
SELECT *
FROM (
    SELECT e.*,
           AVG(salary) OVER (
               PARTITION BY department_id
           ) AS dept_avg
    FROM employee e
) t
WHERE salary > dept_avg;
```

Why `PARTITION BY`?

Instead of one average for the whole table:

```
text
one global average
```

we get:

text

IT average

HR average

Sales average

Real project

Performance analysis by:

text

department

region

team

branch

Q17. Find the highest-paid employee in each department. ☐ ☐ ☐ ☐

sql

```
SELECT *
FROM (
    SELECT e.*,
           ROW_NUMBER() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rn
    FROM employee e
) t
WHERE rn = 1;
```

Important question

What if two employees have exactly the same highest salary?

Use `DENSE_RANK()` if you want all tied employees:

sql

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rn
    FROM employee e
) t
WHERE rn = 1;
```

```

        ) AS rnk
    FROM employee e
) t
WHERE rnk = 1;

```

This distinction impresses interviewers.

Q18. Find the top 3 salaries in each department. □□□□

```

sql
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM employee e
) t
WHERE rnk <= 3;

```

Key concept

This is **Top N per group**.

Remember:

```

text
GROUP BY

```

is not enough.

You typically need:

```

text
Window function
+
PARTITION BY

```

Q19. How do you find employees who don't belong to any department?

Suppose:

text

```
employee.department_id
```

can be NULL.

sql

```
SELECT *  
FROM employee  
WHERE department_id IS NULL;
```

If using a department table

sql

```
SELECT e.*  
FROM employee e  
LEFT JOIN department d  
      ON e.department_id = d.id  
WHERE d.id IS NULL;
```

This finds employees whose department reference does not have a matching department.

Interview trap

Never write:

sql

```
department_id = NULL
```

Use:

sql

```
department_id IS NULL
```

Q20. `COUNT(\*)` vs `COUNT(column)`? ☐ ☐

`COUNT(\*)`

Counts rows.

sql

```
SELECT COUNT(*)  
FROM employee;
```

`COUNT(column)`

Counts non-NULL values.

```
sql
SELECT COUNT(manager_id)
FROM employee;
```

Suppose:

```
text
manager_id

10
20
NULL
30
```

Then:

```
text
COUNT(*)          = 4
COUNT(manager_id) = 3
```

Interview trap

`COUNT(column)` does not count NULL values.

PART C — SUBQUERIES, CTEs & WINDOW FUNCTIONS

Q21–Q30

Q21. What is a subquery?

Short answer

A subquery is a query nested inside another query.

Example:

```
sql
SELECT *
FROM employee
WHERE salary > (
    SELECT AVG(salary)
```

```
FROM employee
);
```

Types

Conceptually:

```
text
Scalar subquery
Single-row subquery
Multi-row subquery
Correlated subquery
```

Q22. What is a correlated subquery? ☐☐☐

A correlated subquery references the outer query.

Example:

```
sql
SELECT e1.*
FROM employee e1
WHERE salary > (
    SELECT AVG(e2.salary)
    FROM employee e2
    WHERE e2.department_id = e1.department_id
);
```

The inner query depends on:

```
text
e1.department_id
```

from the outer query.

Mental model

```
text
Outer employee
    ↓
Find that employee's department average
    ↓
Compare salary
```


Performance note

A correlated subquery may be more expensive depending on the optimizer and query design.

Q23. What is a CTE?

CTE = **Common Table Expression**.

Example:

```
sql
WITH high_salary AS (
    SELECT *
    FROM employee
    WHERE salary > 100000
)
SELECT *
FROM high_salary;
```

Why use CTEs?

They improve readability for complex queries.

Instead of deeply nested subqueries:

```
text
query
  ↓
query
  ↓
query
```

you can structure:

```
sql
WITH step1 AS (...),
    step2 AS (...),
    step3 AS (...)
SELECT ...
FROM step3;
```

Advanced use

Recursive CTEs are useful for hierarchical data such as organizational structures.

Q24. What is a window function? ☐☐☐☐

A window function calculates a value across related rows **without collapsing them into one row**.

Example:

```
sql
SELECT
    name,
    department_id,
    salary,
    AVG(salary) OVER (
        PARTITION BY department_id
    ) AS dept_avg
FROM employee;
```

Every employee remains in the result.

Compare:

```
sql
GROUP BY department_id
```

which produces one row per department.

Key difference

```
text
GROUP BY
→ reduces rows

WINDOW FUNCTION
→ keeps rows
```

Q25. What is `PARTITION BY`?

`PARTITION BY` divides result rows into logical groups for a window function.

Example:

```
sql
AVG(salary) OVER (
    PARTITION BY department_id
)
```

means:

text

```
IT employees → one logical partition
HR employees → another
Sales employees → another
```

Important

PARTITION BY is not the same as **GROUP BY**.

text

```
GROUP BY
→ collapses rows

PARTITION BY
→ does not collapse rows
```

Q26. Find the second-highest salary in each department. ☐☐☐☐

sql

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM employee e
) t
WHERE rnk = 2;
```

Interview value

This combines three concepts:

text

```
Window Function
+
PARTITION BY
+
DENSE_RANK
```

Q27. Find the latest order for each customer. ☐☐☐

Suppose:

```
text
orders
-----
id
customer_id
created_at
amount
```

Use:

```
sql
SELECT *
FROM (
    SELECT o.*,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY created_at DESC
           ) AS rn
    FROM orders o
) t
WHERE rn = 1;
```

Why not `MAX(created\_at)`?

Because you may need other columns from the same row:

```
text
order_id
amount
status
payment_method
```

The window-function approach keeps the complete row.

Q28. Find customers with more than 3 orders.

```
sql
SELECT customer_id,
```

```
        COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Follow-up

Find the actual customer details:

```
sql
SELECT c.*
FROM customer c
JOIN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    HAVING COUNT(*) > 3
) x
ON c.id = x.customer_id;
```

Q29. Find the department with the highest average salary.

```
sql
SELECT department_id,
       AVG(salary) AS avg_salary
FROM employee
GROUP BY department_id
ORDER BY avg_salary DESC
FETCH FIRST 1 ROW ONLY;
```

Database-specific syntax may also be:

```
sql
LIMIT 1
```

Alternative

```
sql
WITH dept_avg AS (
    SELECT department_id,
           AVG(salary) AS avg_salary
```

```

        FROM employee
        GROUP BY department_id
    )
    SELECT *
    FROM dept_avg
    WHERE avg_salary = (
        SELECT MAX(avg_salary)
        FROM dept_avg
    );

```

The second version can return ties.

Q30. How do you find gaps in sequential IDs?

Suppose IDs should be:

```

text
1
2
3
4
5

```

but database contains:

```

text
1
2
4
5

```

Potential missing value:

```

text
3

```

A simple technique:

```

sql

SELECT t.id + 1 AS missing_id
FROM orders t
LEFT JOIN orders t2

```

```
ON t2.id = t.id + 1
WHERE t2.id IS NULL;
```

But this may also identify the value after the current maximum, so production logic often needs explicit bounds.

Better interview answer

Finding gaps depends on whether the IDs are guaranteed sequential and what range should be considered. I would define the expected range before writing the final query.

That is stronger than blindly giving a query.

PART D — INDEXES, PERFORMANCE & QUERY OPTIMIZATION

Q31–Q40

Q31. What is an index? □□□□□

Short answer

An index is a data structure that helps the database locate rows faster without scanning the entire table.

Example:

```
sql
CREATE INDEX idx_employee_email
ON employee(email);
```

Without a useful index:

```
text
Query
↓
scan many rows
↓
find matching email
```

With index:

```
text
Query
↓
Index lookup
```

```
↓  
matching rows
```

Real project

For:

```
sql  
  
SELECT *  
  
FROM users  
  
WHERE email = ?;
```

an index on:

```
text  
  
email
```

can dramatically improve lookup performance for suitable workloads.

Q32. What are the disadvantages of indexes? ☐ ☐

Indexes are not free.

They consume:

```
text  
  
Disk space  
  
Memory/cache  
  
Write overhead  
  
Maintenance cost
```

When:

```
sql  
  
INSERT  
  
UPDATE  
  
DELETE
```

occurs, indexes may also need updates.

Interview answer

Indexes improve read performance but add storage and write-maintenance overhead, so I avoid creating indexes indiscriminately.

Q33. What is a composite index? ☐☐☐

A composite index uses multiple columns.

```
sql
CREATE INDEX idx_orders_customer_status
ON orders(customer_id, status);
```

Important concept

Column order matters.

For:

```
text
(customer_id, status)
```

the index is typically more useful for queries beginning with:

```
sql
WHERE customer_id = ?
```

or:

```
sql
WHERE customer_id = ?
AND status = ?
```

than for a query using only:

```
sql
WHERE status = ?
```

This connects to the idea commonly called the **leftmost-prefix principle** for B-tree indexes.

Q34. What is a covering index?

A covering index contains all columns needed by a query so the database may be able to satisfy it directly from the index.

Example:

```
sql
CREATE INDEX idx_user_email_name
ON users(email, name);
```

Query:

```
sql
SELECT email, name
FROM users
WHERE email = 'a@example.com';
```

The index already contains:

```
text
email
name
```

so the database may avoid looking up the base table row, depending on the engine and execution plan.

Strong answer

Don't say:

"Covering index always avoids table access."

Say:

"It can allow the optimizer to satisfy the query from the index alone when the execution plan supports it."

Q35. What is a clustered index vs non-clustered index?

The exact implementation depends on the database engine.

Conceptually:

Clustered

The table's row storage is organized around the clustering key.

Non-clustered

A separate index structure points to the relevant rows.

Interview trap

Do not memorize:

"Every database has exactly one clustered index."

That is database-specific.

Best answer

Clustered and non-clustered indexing describe how indexed data relates to physical or logical row storage, but the exact implementation is database-engine specific.

Q36. How do you analyze a slow SQL query? ☐☐☐☐☐

This is a major product-company question.

Don't say:

"Add an index."

Use a systematic approach.

Step 1

Inspect execution plan.

For example:

```
sql
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 100;
```

Some databases provide:

```
sql
EXPLAIN ANALYZE
```

Step 2

Check:

```
text
Full table scans
Index usage
Join strategy
Estimated vs actual rows
Sort operations
Temporary tables
Filter selectivity
```

Step 3

Check indexes.

Step 4

Check joins.

Step 5

Check row counts and data distribution.

Step 6

Check whether application code causes repeated queries.

For example:

```
text
N+1 queries
```

Step 7

Measure before and after.

Strong interview answer

I first reproduce and measure the slow query, inspect the execution plan, identify whether the bottleneck is scanning, joins, sorting, poor cardinality estimates, locking, or data-access behavior, then make the smallest evidence-based change and benchmark it.

Q37. What is a full table scan?

A full table scan means the database examines many or all rows of a table to evaluate a query.

Example:

```
sql
SELECT *
FROM employee
WHERE salary > 100000;
```

If no useful index exists, the optimizer may choose a scan.

Is full table scan always bad?

No.

☐ Important interviewer trap.

For a small table, a full scan may be faster than an index lookup.

Best answer

A full table scan is not inherently bad. Whether it is appropriate depends on table size, selectivity, statistics, caching and the optimizer's cost model.

Q38. What is query selectivity?

Selectivity describes how effectively a predicate narrows the result set.

Example:

```
text
WHERE user_id = 123
```

may return:

```
text
1 row
```

Very selective.

But:

```
sql
WHERE gender = 'M'
```

may return:

```
text
50% of table
```

Much less selective.

Why important?

Highly selective predicates often make indexes more useful.

Q39. Why can `SELECT \*` be a bad practice? ☐ ☐

Example:

```
sql
SELECT *
FROM orders;
```

Problems include:

text

```
Unnecessary columns  
More network transfer  
More memory  
Potentially worse index coverage  
Coupling to schema changes
```

Prefer:

sql

```
SELECT id, status, amount  
FROM orders;
```

Real Spring Boot example

An API may return:

json

```
{  
  "id": 101,  
  "status": "PAID",  
  "amount": 500  
}
```

There is no reason to retrieve 30 database columns if the API needs 3.

Q40. What is the N+1 query problem from the SQL perspective? ☐☐☐☐

Suppose you load:

text

```
100 orders
```

Then execute one query for each order's customer.

You could accidentally generate:

text

```
1 query  
+  
100 queries  
=  
101 queries
```

Example pattern:

```
text

SELECT * FROM orders;

SELECT * FROM customer WHERE id = 1;
SELECT * FROM customer WHERE id = 2;
SELECT * FROM customer WHERE id = 3;
...
```

Better approach

Use a join where appropriate:

```
sql

SELECT o.id,
       o.amount,
       c.name
FROM orders o
JOIN customer c
      ON c.id = o.customer_id;
```

In JPA, this also relates to:

```
text

fetch join
EntityGraph
batch fetching
DTO projections
```

PART E — TRANSACTIONS & CONCURRENCY

Q41–Q45

Q41. What is a transaction? ☐☐☐☐

A transaction is a logical unit of database work that should satisfy the transaction's consistency guarantees.

Example:

Bank transfer:

```
text
Debit Account A
+
Credit Account B
```

Both should succeed together.

```
text
BEGIN
    debit
    credit
COMMIT
```

If something fails:

```
text
ROLLBACK
```

ACID

```
text
A → Atomicity
C → Consistency
I → Isolation
D → Durability
```

This is especially important because your earlier JPA/Hibernate module and microservices module already built heavily around transactions and distributed consistency.

Q42. Explain ACID. ☐☐☐☐☐

Atomicity

All operations succeed or the transaction rolls back.

Consistency

The database moves from one valid state to another valid state according to its constraints and transaction rules.

Isolation

Concurrent transactions should not improperly interfere according to the chosen isolation level.

Durability

Once committed, data should survive subsequent failures according to the database's durability guarantees.

Example

```
text
Transfer ₹100

A = 1000
B = 500

Debit A → 900
Credit B → 600
```

Both actions belong to one transaction.

Q43. What are transaction isolation levels? ☐☐☐☐☐

Common SQL isolation levels:

```
text
READ UNCOMMITTED
READ COMMITTED
REPEATABLE READ
SERIALIZABLE
```

They control which effects of concurrent transactions can be observed.

READ UNCOMMITTED

Lowest isolation.

Can allow dirty reads depending on the engine.

READ COMMITTED

A transaction reads only committed data.

REPEATABLE READ

Repeated reads of the same row generally remain stable within the transaction, but exact phantom behavior is database-specific.

SERIALIZABLE

Strongest standard isolation level, aiming to provide behavior equivalent to serial execution, often at higher concurrency cost.

Interview trap

Don't claim:

"Isolation levels work identically in every database."

They don't.

Database implementation matters.

Q44. What are dirty reads, non-repeatable reads and phantom reads?

□ □ □ □

Dirty read

Transaction A sees uncommitted changes from transaction B.

```
text
B updates
↓
not committed
↓
A reads it
```

If B rolls back, A saw data that never actually committed.

Non-repeatable read

Transaction A reads a row twice and gets different committed values because another transaction changed it between the reads.

Phantom read

Transaction A repeats a range query and sees new or removed rows matching the condition.

Example:

First:

```
sql
SELECT *
FROM employee
WHERE salary > 50000;
```

returns:

```
text
10 rows
```

Another transaction inserts a matching employee.

Repeat:

```
sql
SELECT *
FROM employee
WHERE salary > 50000;
```

Now:

```
text
11 rows
```

That new row is a phantom.

Q45. Optimistic vs pessimistic locking in SQL? ☐☐☐

Optimistic locking

Assumes conflicts are relatively uncommon.

Common JPA implementation:

```
java
@Version
private Long version;
```

Conceptually SQL may become:

```
sql
UPDATE product
SET stock = ?, version = version + 1
WHERE id = ?
AND version = ?;
```

If zero rows are updated:

```
text
someone changed the row
```

Pessimistic locking

The database locks the row to prevent conflicting concurrent modifications.

Example:

```
sql
SELECT *
FROM product
WHERE id = 10
FOR UPDATE;
```

Exact syntax depends on the database.

Interview answer

Optimistic locking detects conflicts when updating, while pessimistic locking acquires database locks to prevent conflicting access during the transaction.

This directly connects to the JPA/Hibernate material from Module 3.

PART F — ADVANCED SQL INTERVIEW QUESTIONS

Q46–Q50

Q46. Difference between `DELETE`, `TRUNCATE`, and `DROP`?

□ □ □ □

DELETE

Removes rows.

```
sql
DELETE FROM employee
WHERE department_id = 10;
```

Can normally use a `WHERE` clause.

TRUNCATE

Removes all rows from a table.

```
sql
TRUNCATE TABLE employee;
```

Its behavior regarding logging, rollback, identity counters and locking is database-specific.

DROP

Removes the table object itself.

```
sql
DROP TABLE employee;
```

Conceptually:

```
text
DELETE
→ remove selected rows

TRUNCATE
→ remove table data

DROP
→ remove table definition/object
```

Interview trap

Do not oversimplify transaction behavior across all databases. `TRUNCATE` semantics are vendor-specific.

Q47. What is normalization? ☐☐☐

Normalization is the process of organizing relational data to reduce unnecessary duplication and update anomalies.

Common forms:

```
text
1NF
2NF
3NF
BCNF
```

Example of bad design

```
text
order_id
customer_name
customer_email
product_name
product_price
```

If a customer's email changes, many rows may need updates.

Instead:

```
text
customer
-----
id
name
email

orders
-----
id
customer_id
```

Benefits

```
text
Less duplication
Better consistency
Cleaner updates
```

Trade-off

Over-normalization can lead to:

```
text
More joins
More query complexity
Potential read overhead
```

This leads to an important product-company concept:

Normalize by default, denormalize intentionally.

Q48. What is denormalization and when would you use it? ☐☐☐

Denormalization intentionally duplicates data to improve read performance or simplify access patterns.

Example:

Instead of joining:

```
text
orders
+
customer
```

you may store:

```
text
orders
customer_name_snapshot
```

Why?

For workloads where:

```
text
reads >>> writes
```

and the duplicated data is acceptable.

Real project example

An order may intentionally keep:

```
text
customer_name
shipping_address
```

as historical snapshots even if customer information later changes.

This is not necessarily a bad design.

Tricky interviewer question

Does denormalization always make queries faster?

No.

It can introduce:

```
text
update anomalies
storage overhead
consistency challenges
```

You should denormalize based on measured access patterns.

Q49. What is a deadlock? □□□□□

A deadlock occurs when transactions wait for locks held by each other, creating a cycle.

Example:

```
text
Transaction A
locks Row 1
    ↓
waits for Row 2

Transaction B
locks Row 2
    ↓
waits for Row 1
```

Diagram:

```
text
A → waits for B
↑       ↓
└────────┘
```

How can you reduce deadlocks?

Use consistent locking order.

For example, always lock:

```
text
Account A
then Account B
```

rather than sometimes:

```
text
A → B
```

and elsewhere:

```
text
B → A
```

Other techniques:

text

```
Shorter transactions
Proper indexes
Smaller lock scope
Consistent ordering
Appropriate isolation
Retry after deadlock where safe
```

Real Spring Boot scenario

Two concurrent order/payment operations can accidentally lock rows in different orders.

The database may detect the deadlock and abort one transaction.

Strong answer

I wouldn't simply increase timeouts. I would inspect the deadlock graph, identify the conflicting lock order, reduce transaction scope and make lock acquisition order consistent.

Q50. Design the SQL for an e-commerce Order system. □□□□□

This is the **master SQL question** for the module, similar to the production-design questions at the end of previous modules.

Suppose interviewer says:

"Design the database tables for an e-commerce order system and explain the relationships, constraints, indexes and important queries."

Step 1 — Tables

Customer

sql

```
CREATE TABLE customer (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(200) NOT NULL UNIQUE,
    created_at TIMESTAMP NOT NULL
);
```

Product

sql

```
CREATE TABLE product (
```

```
    id BIGINT PRIMARY KEY,  
    name VARCHAR(200) NOT NULL,  
    price DECIMAL(12,2) NOT NULL,  
    stock INT NOT NULL,  
    created_at TIMESTAMP NOT NULL  
);
```

Orders

```
sql  
  
CREATE TABLE orders (  
    id BIGINT PRIMARY KEY,  
    customer_id BIGINT NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    total_amount DECIMAL(12,2) NOT NULL,  
    created_at TIMESTAMP NOT NULL,  
  
    FOREIGN KEY (customer_id)  
        REFERENCES customer(id)  
);
```

Order Item

```
sql  
  
CREATE TABLE order_item (  
    id BIGINT PRIMARY KEY,  
    order_id BIGINT NOT NULL,  
    product_id BIGINT NOT NULL,  
    quantity INT NOT NULL,  
    unit_price DECIMAL(12,2) NOT NULL,  
  
    FOREIGN KEY (order_id)  
        REFERENCES orders(id),  
  
    FOREIGN KEY (product_id)  
        REFERENCES product(id)  
);
```

Relationship

text

Customer

|

| 1

|

| N

Orders

|

| 1

|

| N

OrderItem

|

| N

|

| 1

Product

Step 2 — Important indexes

For orders by customer:

sql

```
CREATE INDEX idx_orders_customer  
ON orders(customer_id);
```

For customer order history:

sql

```
CREATE INDEX idx_orders_customer_created  
ON orders(customer_id, created_at DESC);
```

For order items:

sql

```
CREATE INDEX idx_order_item_order  
ON order_item(order_id);
```

For product lookup:

```
sql
CREATE INDEX idx_order_item_product
ON order_item(product_id);
```

The exact index set should be determined from real queries and execution plans rather than created blindly.

Step 3 — Find customer's recent orders

```
sql
SELECT id,
       status,
       total_amount,
       created_at
FROM orders
WHERE customer_id = ?
ORDER BY created_at DESC;
```

Potentially supported by:

```
text
(customer_id, created_at)
```

index.

Step 4 — Find order details

```
sql
SELECT
    o.id AS order_id,
    o.status,
    o.total_amount,
    p.id AS product_id,
    p.name,
    oi.quantity,
    oi.unit_price
FROM orders o
JOIN order_item oi
    ON oi.order_id = o.id
JOIN product p
```

```
ON p.id = oi.product_id
WHERE o.id = ?;
```

Step 5 — Calculate customer spending

```
sql
SELECT customer_id,
       SUM(total_amount) AS total_spent
FROM orders
WHERE status = 'COMPLETED'
GROUP BY customer_id;
```

Step 6 — Top customers

```
sql
SELECT customer_id,
       SUM(total_amount) AS total_spent
FROM orders
WHERE status = 'COMPLETED'
GROUP BY customer_id
ORDER BY total_spent DESC
FETCH FIRST 10 ROWS ONLY;
```

Step 7 — Most purchased products

```
sql
SELECT product_id,
       SUM(quantity) AS total_quantity
FROM order_item
GROUP BY product_id
ORDER BY total_quantity DESC
FETCH FIRST 10 ROWS ONLY;
```

Step 8 — Transaction for creating an order

Conceptually:

```
text
BEGIN TRANSACTION
```

1. Create order
2. Create order items
3. Decrease inventory
4. Validate business constraints
5. COMMIT

For concurrent inventory updates, you must also consider:

```
text
optimistic locking
or
pessimistic locking
```

For example:

```
sql
SELECT stock
FROM product
WHERE id = ?
FOR UPDATE;
```

Then:

```
sql
UPDATE product
SET stock = stock - ?
WHERE id = ?;
```

The exact strategy depends on concurrency requirements and the database.

☐ VERY IMPORTANT PRODUCT-COMPANY SQL TRAPS

Before moving to Module 8, make sure these are absolutely solid:

1. Second-highest salary

```
sql
SELECT MAX(salary)
FROM employee
WHERE salary < (
    SELECT MAX(salary)
```

```
FROM employee
);
```

2. Duplicate records

```
sql
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

3. Top N per department

```
sql
DENSE_RANK() OVER (
    PARTITION BY department_id
    ORDER BY salary DESC
)
```

4. Employees above department average

```
sql
AVG(salary) OVER (
    PARTITION BY department_id
)
```

5. Customers without orders

```
sql
LEFT JOIN
+
WHERE right.id IS NULL
```

6. Latest row per group

```
sql
ROW_NUMBER() OVER (
    PARTITION BY ...
    ORDER BY created_at DESC
)
```

7. Duplicate-safe ranking

```
text
ROW_NUMBER
RANK
DENSE_RANK
```

8. Slow-query reasoning

```
text
EXPLAIN
→ execution plan
→ scans
→ joins
→ indexes
→ cardinality
→ sorting
→ locks
→ application behavior
```

9. Transaction reasoning

```
text
ACID
+
Isolation
+
Locking
+
Deadlocks
```

10. Backend interview connection

Always connect SQL back to:

```
text
Spring Data JPA
Hibernate
@Transactional
N+1
Pagination
Indexes
Lazy/Eager loading
Optimistic locking
```



```
Pessimistic locking
Microservices
Database-per-Service
```

That preserves the progression of the earlier modules: the microservices material explicitly emphasized independent databases, consistency, failures, and transaction design, while the security material continued the same production-oriented interview style.

Module 7 completion map

text

```
Q1  SQL basics
Q2  WHERE vs HAVING
Q3  Primary Key
Q4  Primary vs UNIQUE
Q5  Foreign Key
Q6  JOIN
Q7  INNER vs LEFT JOIN
Q8  Self JOIN
Q9  CROSS JOIN
Q10 GROUP BY

Q11 Second highest salary
Q12 Nth highest salary
Q13 RANK vs DENSE_RANK vs ROW_NUMBER
Q14 Duplicate records
Q15 Above average salary
Q16 Above department average
Q17 Highest salary per department
Q18 Top 3 per department
Q19 NULL / missing department
Q20 COUNT(*) vs COUNT(column)

Q21 Subquery
Q22 Correlated subquery
Q23 CTE
Q24 Window function
Q25 PARTITION BY
Q26 Second highest per department
Q27 Latest order per customer
```

Q28 Customers with >3 orders
Q29 Highest-average department
Q30 Gaps in IDs

Q31 Index
Q32 Index drawbacks
Q33 Composite index
Q34 Covering index
Q35 Clustered vs non-clustered
Q36 Slow query analysis
Q37 Full table scan
Q38 Selectivity
Q39 SELECT *
Q40 N+1

Q41 Transaction
Q42 ACID
Q43 Isolation levels
Q44 Dirty/non-repeatable/phantom reads
Q45 Optimistic/pessimistic locking
Q46 DELETE/TRUNCATE/DROP
Q47 Normalization
Q48 Denormalization
Q49 Deadlocks
Q50 Production e-commerce SQL design

Module 7 is therefore complete: 50 SQL interview questions + practical queries.

The next progression in the established Bible is **Module 8 — 50 Multithreading questions**, where the same format will move from Java concurrency fundamentals into `synchronized`, `volatile`, locks, executors, `CompletableFuture`, atomics, thread pools, deadlocks, race conditions, concurrent collections, virtual threads, and production concurrency scenarios.